الكلية متعددة التخصصات - ورزازات

FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

Pr. OMARI Kamal

2025/2026

الكلية متعددة التخصصات - ورزازات
ⵜⴰⵖⵉⵡⴰⵏⵜ ⵜⴰⴳⵜⵉⵎⵓⵜⵜⵉⵜ – ⵡⵓⵔⵣⵣⴰⵣⴰⵜ
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE
جامعة ابن زهر - أكادير
UNIVERSITÉ IBN ZOHR - AGADIR

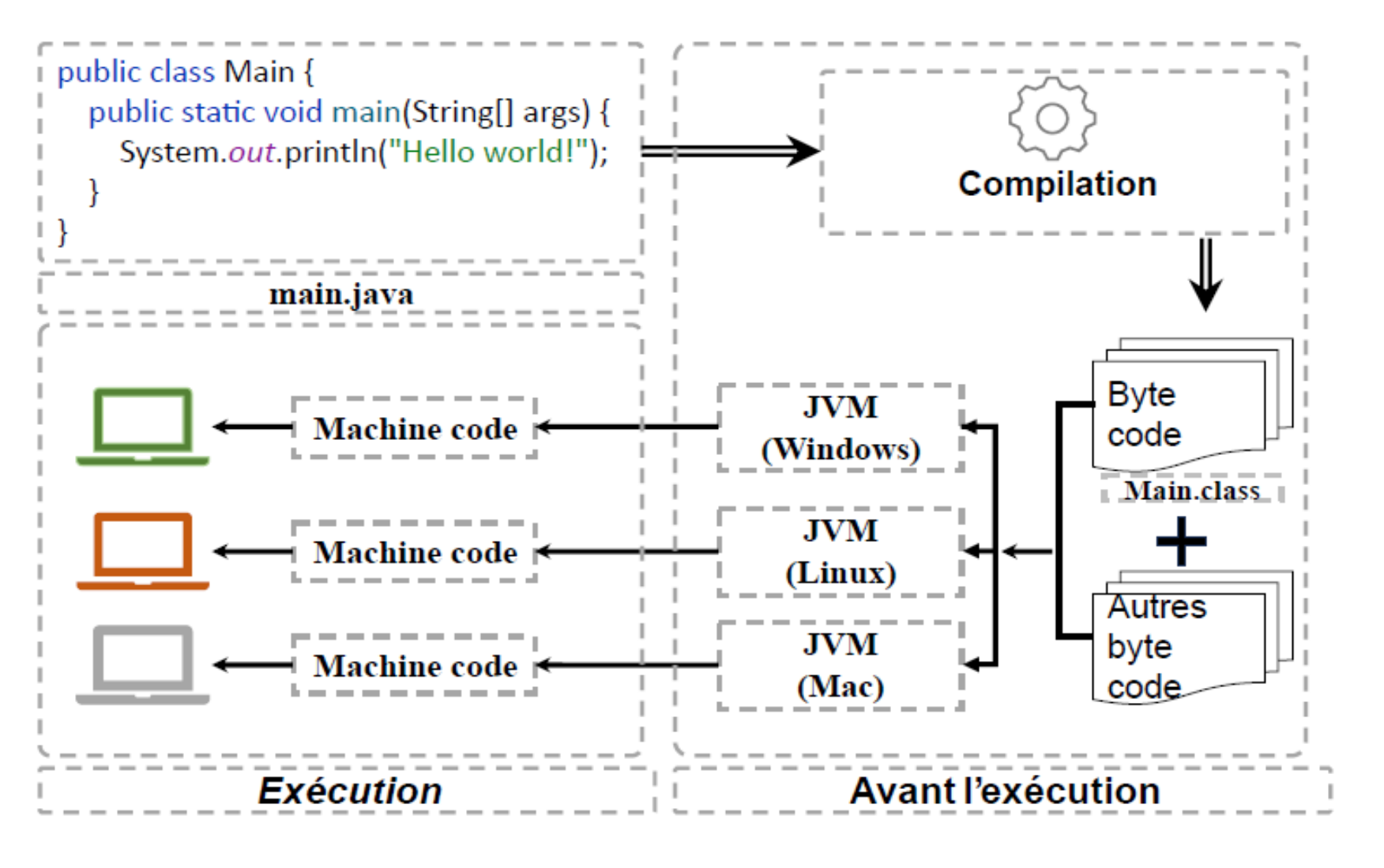
Partie 1

Introduction au Java

Caractéristiques du langage java

- **Orienté objet**: Java repose sur le paradigme de la programmation orientée objet (POO).
- **Portable**: Grâce à la machine virtuelle Java (JVM), le code Java peut s'exécuter sur différentes plateformes ("Write Once, Run Anywhere").
- **Sécurisé**: Java propose des fonctionnalités avancées pour la gestion de la sécurité et la protection des applications.
- **Multithreading**: Java permet l'exécution simultanée de plusieurs threads, optimisant ainsi la performance.

Mode d'exécution



Pourquoi Java?

L’API de Java

• Java fournit de nombreuses librairies de classes remplissant des fonctionnalités très diverses: c’est l’API Java.

• API (Application and Programming Interface /Interface pour la programmation d’applications): Ensemble de bibliothèques permettant une programmation plus aisée.

• Ces classes sont regroupées par catégories en paquetages (ou "packages").

Pourquoi Java?

L'API de Java

Les principaux paquetages:

- java.util : structures de données classiques.
- java.io: entrées / sorties.
- java.lang : chaînes de caractères, interaction avec l'OS, threads.
- java.applet : les applets sur le web.
- java.awt : interfaces graphiques, images et dessins...
- javax.swing : package récent proposant des composants « légers» pour la création d'interfaces graphiques.
- java.net : sockets, URL ...
- java.rmi : Remote Method Invocation.
- java.sql : fournit le package JDBC.

Outil de développement: le JDK

- JDK signifie Java Development Kit (Kit de développement Java).
- Il contient:
  - les classes de base de l'API java,
  - le compilateur: **javac,**
  - la JVM (machine virtuelle): **java,**
  - le générateur de documentation au format HTML : **javadoc.**

Partie 2

Structures fondamentales

Les commentaires (1)

- /* commentaire sur une ou plusieurs lignes */
- // commentaire de fin de ligne
- /** commentaire d'explication */

- Les commentaires d'explication se placent généralement juste avant une déclaration (d'attribut ou de méthode).
- Ils sont récupérés par l'utilitaire javadoc et inclus dans la documentation ainsi générée.

Les commentaires (2)

```
/**
 * Classe simple pour effectuer une addition de deux nombres.
 * @author Test
 * @version 1.0
 */
public class Calculatrice {

   /**
    * Calcule la somme de deux entiers.
    * @param a premier entier
    * @param b second entier
    * @return la somme de a et b
    */
   public int addition(int a, int b)
      { return a + b;
   }
}
```

javadoc Calculatrice.java

Classe Calculatrice
Description :
Classe simple pour effectuer une addition de deux nombres.
Author : Test
Version : 1.0

Méthode	Description
int addition(int a, int b)	Calcule la somme de deux entiers. Paramètres : a – premier entier b – second Entier Retourne : la somme de a et b

Instructions, blocs et blancs

- Les instructions Java se terminent par un ;
- Les blocs sont délimités par :
  - {pour le début de bloc.
  - } pour la fin du bloc.
- Un bloc permet de définir un regroupement d'instructions. La définition d'une classe ou d'une méthode se fait dans un bloc.
- Les espaces, tabulations, sauts de ligne sont autorisés. Cela permet de présenter **un code plus lisible.**

Point d'entrée d'un programme Java

- Pour pouvoir faire un programme exécutable il faut toujours une classe qui contienne la méthode **"main"**
- C'est le point d'entrée dans le programme: le microprocesseur sait qu'il va commencer à exécuter les instructions à partir de cet endroit.

```
public static void main(String[] args) {
...
}
```

Identificateurs

• On a besoin de nommer les classes, les variables, les constantes, etc. ;

on parle d'***identificateur***. (Attention: Java distingue les majuscules des minuscules.)

Type d'identificateur	Exemple valide	Remarques
Nom de classe	MaClasse	Commence par une lettre majuscule (convention).
Nom de méthode	calculerValeur	Commence par une lettre minuscule (convention).
Nom de variable	compteur	Commence par une lettre.
Nom de variable	_temperature	Commence par le caractère de soulignement (_).
Nom de variable	$montantFinal	Commence par le signe dollar ($).
Nom de variable	i18n	Peut contenir des chiffres (sauf en première position).
Nom de constante	MAXI	En majuscules(convention).

Affectation

- L'affectation est l'opération qui consiste à donner une **valeur** à un **identificateur** (variable ou constante) que l'on a préalablement déclaré.
- Elle utilise le symbole de l'**égalité simple** (=).
- La **variable** (ou l'identificateur) est toujours placée à **gauche** du signe =.
- La **valeur** ou l'**expression** est toujours placée à **droite** du signe =.

Exemple:

```
int compteur = 0;
compteur= 5; // C'est permis
final int NOMBRE_JOURS = 7; // NOMBRE_JOURS = 8; // <-- C'est interdit, cause une erreur de compilation!
```

Écriture en Java

L'écriture sur l'écran se fait à l'aide de l'objet **System.out** et de ses méthodes principales:

- **System.out.print()**: Affiche le texte **sans retour à la ligne**.

Exemple:

```
System.out.print("Bonjour ");
System.out.print("tout le monde !");
```

- **System.out.println():** Affiche le texte **suivi d'un saut de ligne**.

Exemple:

```
System.out.println("Bonjour");
System.out.println("tout le monde !");
```

- **System.out.printf():** Permet d'afficher du texte **formaté**, similaire à printf en langage C.

Exemple:

```
int age = 20;
System.out.printf("J'ai %d ans.", age);
```

Lecture en Java

- Pour lire des données saisies par l'utilisateur, on utilise la **classe Scanner**, appartenant au paquetage java.util.
- Avant d'utiliser Scanner, il faut l'importer: import java.util.Scanner;
- Création d'un objet Scanner: Scanner sc = new Scanner(System.in);

Exemple:

```
import java.util.Scanner;
public class Lecture {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
System.out.print("Entrez votre nom : ");
String nom = scanner.nextLine();
System.out.println("Bonjour, " + nom + " !");
scanner.close();
}
}
```

Les types de bases

- En Java, tout est objet sauf les types de base.
- Il y a 8 types de base :
  - un type **booléen** pour représenter les variables ne pouvant prendre que 2 valeurs (vrai et faux, 0 ou 1, etc.) : boolean avec les valeurs associées true et false.
  - un type pour représenter les caractères: **char**.
  - quatre types pour représenter les entiers de divers taille : **byte**, **short**, **int** et **long**.
  - deux types pour représenter les réelles : **float** et **double**.

Les opérateurs

Les opérateurs Java sont des symboles spéciaux qui effectuent des opérations sur les variables et les valeurs. Ils sont utilisés pour manipuler les données et les variables dans les expressions. Java propose un large éventail d'opérateurs classés en plusieurs types en fonction de leur fonctionnalité.

- **Opérateurs arithmétiques**

+ (Ajout)

- (Soustraction)

* (Multiplication)

/ (Division)

% (module)

```
int a = 10;
int b = 5;
System.out.println("Addition: " + (a + b));      // Output: 15
System.out.println("Subtraction: " + (a - b));   // Output: 5
System.out.println("Multiplication: " + (a * b));// Output: 50
System.out.println("Division: " + (a / b));      // Output: 2
System.out.println("Modulus: " + (a % b));       // Output: 0
```

Les opérateurs

- **Opérateurs relationnels**

== (égal à)

!= (non égal à)

> (Supérieur à)

< (moins de)

>= (supérieur ou égal)

<= (Inférieur ou égal à)

```
int a = 10;
int b = 5;
System.out.println(a == b); // Output: false
System.out.println(a > b);  // Output: true
System.out.println(a < b);  // Output: false
```

- **Opérateurs logiques**

&& (ET logique)

|| (OU logique)

! (logique NOT)

```
boolean x = true;
boolean y = false;
System.out.println(x && y); // Output: false
System.out.println(x || y); // Output: true
System.out.println(!x);     // Output: false
```

Les opérateurs

- **Opérateurs d'assignation**

+= (Ajouter et attribuer)

-= (Soustraire et céder)

*= (Multiplier et attribuer)

/= (Diviser pour mieux régner)

%= (Modulus et assign)

```
int a = 10;
a += 5;  // Equivalent to a = a + 5
System.out.println(a); // Output: 15
```

- **Opérateurs d'incrémentation**

++ (Incrément)

-- (Diminution)

```
int a = 10;
System.out.println(++a); // Output: 11
System.out.println(--a); // Output: 10
```

Instructions conditionnelles

- Effectuer une ou plusieurs instructions seulement si une certaine condition est vérifiée.

```
if (condition){
//bloc d'instructions
}
```

- Effectuer une ou plusieurs instructions si une certaine condition est vérifiée sinon effectuer d'autres instructions.

```
if (condition) {
//1er bloc d'instructions
}
else {
//2ème bloc d'instructions
}
```

Instructions conditionnelles

- ***Exemple***

```
import java.util.Scanner;
public class Comparaison {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);    // Création de l'objet Scanner
      System.out.print("Entrez le premier entier (a) : ");
      int a = sc.nextInt();                   // Lecture du premier entier
      System.out.print("Entrez le deuxième entier (b) : ");
      int b = sc.nextInt();                   // Lecture du deuxième entier
      if (a > b)
         System.out.println("Le plus grand est : " + a);
      else
         System.out.println("Le plus grand est : " + b);

      sc.close();                             // Fermeture du Scanner
   }
}
```

Instructions conditionnelles

- **switch**

• L'utilisation de if / else peut s'avérer lourde quand on doit traiter plusieurs sélections et de multiples alternatives.

• pour cela existe en Java le switch / case assez identique à celui de C/C++ .

• La valeur sur laquelle on teste doit être un char ou un entier.

• L'exécution des instructions correspondant à une alternative commence au niveau du case correspondant et se termine à la rencontre d'une instruction break ou arrivée à la fin du switch.

```
switch(nb)
 {
case 1:
System.out.println("Un");
break;
case 2:
System.out.println("Deux");
break;
default:
System.out.println("Autrenombre");
break;
}
```

Les structures répétitives

- On veut répéter une ou plusieurs instructions un nombre indéterminés de fois: on répète l'instruction ou le bloc d'instruction tant que une certaine condition reste vraie.
- nous avons en Java une première boucle while:

```
• while (condition)
{
//bloc d'instructions
}
```

```
import java.util.Scanner;
public class Factorielle {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Entrer n: ");
      int n = sc.nextInt();
      int result = 1, i = n;
      while (i > 1) {
         result = result * i;
         i--;  }
      System.out.println("La factorielle de " + n + " vaut " + result);
      sc.close();}}
```

Les structures répétitives

- Un autre type de boucle:

  - do (condition) {
  //bloc d'instructions
  }while (condition);

```
int i = 0;
do {
System.out.println(i);
i++;
}while (i < 5);
```

Les structures répétitives

Boucle déterminée : "for"

• On veut répéter une ou plusieurs instructions un nombre déterminés de fois : on répète l'instruction ou le bloc d'instructions pour un certain nombre de pas.

```
for (int i = 1; i <= 10; i++)
{
System.out.println(i);
}
```

Les structures répétitives

Boucle déterminée : "for each"

- for-each (aussi appelé enhanced for loop) est une version simplifiée de la boucle for qui permet d'itérer directement sur les éléments d'un tableau ou d'une collection (par exemple, ArrayList, HashSet, etc.) sans avoir à gérer un compteur ou un index.

- **Syntaxe générale:**

```
for (Type element : collectionOuTableau) {
// Instructions utilisant 'element'
}
```

- Type : le type des éléments contenus dans la collection ou le tableau.
- Elément : variable temporaire qui prend successivement la valeur de chaque élément.
- collection Ou Tableau : tableau ou objet qui implémente l'interface Iterable (par ex. ArrayList, HashSet, etc.).

Les structures répétitives

Boucle déterminée : “for each”

- **Exemple:**

```
public static void main(String[] args){
int collection[] = {10, 20, 30, 40, 50}
for (int value : collection){
System.out.println(value);
}
}
```

Partie 3

Tableaux

Les tableaux

- Les tableaux permettent de stocker plusieurs valeurs de même type.
  - Les valeurs contenues dans la variable sont repérées par un indice.
  - Les tableaux sont des objets
- Déclaration : (Type [] nomTableau ;).

Exemple: int notes[];

int[] notes; // forme recommandée en Java

- Création du tableau

notes = new int[5]; // crée un tableau de 5 entiers initialisés à 0

- Déclaration + création en une seule ligne

int[] notes = new int[5];

➢ ou bien, avec initialisation directe: int[] notes = {12, 15, 18, 10, 14};

Les tableaux: *Opérations de Base*

- **Parcourir un tableau**
  - Pour parcourir un tableau, on utilise l'instruction "for"

```
Méthode 1:
public class Demo {
public static void main( String [] args ) {
// Déclaration et initialisation du tableau
int array [] = { 10, 20, 30, 40, 50 };
// Parcours du tableau
for ( int i=0; i<array.length;i++ ) {
System.out.println( array[ i ] );
}
}
}
```

Les tableaux: *Opérations de Base*

- **Parcours avec un for each**

```
Méthode 2:
public class Demo {
public static void main( String [] args ) {
// Declaration et initialisation du tableau
int array[]  = { 10, 20, 30, 40, 50 };
System.out.println( "Parcours du tableau avec un for each" );
for ( int value : array ) {
System.out.println( value );
}
}
}
```

Les tableaux: *Opérations de Base*

- **Accéder à un élément**

Exemple:

```
int[] notes = {12, 15, 18, 10, 14};
// Afficher le premier élément
System.out.println(notes[0]);    // Résultat : 12
// Afficher le troisième élément
System.out.println(notes[2]);    // Résultat : 18
```

- **Modifie un élément**

```
notes[1] = 20;   // Remplace 15 par 20
notes[3] = 11;   // Remplace 10 par 11
```

- **Obtenir la Longueur**

```
int longueur = tableau.length;
```

Arrays en java

- Les tableaux offrent les **opérations essentielles**:
  - accès à un élément,
  - modification,
  - calcul de la longueur.
- Si l'on veut par exemple :
  - trier un tableau,
  - le copier,
  - le comparer,

➢ Il serait **fastidieux** de tout réécrire à chaque fois.

➢ Pour simplifier ces manipulations, Java fournit une classe utilitaire appelée **Arrays,** qui contient de nombreuses méthodes prêtes à l'emploi pour travailler efficacement avec les tableaux.

Arrays en java

- Arrays.toString() → **Afficher le contenu d'un tableau**

```
import java.util.Arrays;
int[] notes = {12, 8, 15, 10, 18};
System.out.println(Arrays.toString(notes));
```

→ [12, 8, 15, 10, 18]

- Arrays.fill() → **Remplir un tableau avec une même valeur**

```
import java.util.Arrays;
int[] tab = new int[5];
Arrays.fill(tab, 7);     // Remplit tout le tableau avec 7
System.out.println(Arrays.toString(tab));
```

→ [7, 7, 7, 7, 7]

Arrays en java

- Arrays.sort() → **Trier un tableau en ordre croissant**

```
import java.util.Arrays;
int[] valeurs = {5, 2, 8, 1, 3};
Arrays.sort(valeurs);
System.out.println(Arrays.toString(valeurs));
```

→ [1, 2, 3, 5, 8]

- Arrays.equals() → **Comparer deux tableaux**

```
import java.util.Arrays;
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
int[] c = {3, 2, 1};
System.out.println(Arrays.equals(a, b));  // true
System.out.println(Arrays.equals(a, c));  // false
```

- **Exercice d'application:**
  - Écrire un programme Java qui:
    - **Demande à l'utilisateur** de saisir les **5 valeurs** du tableau A.
    - **Demande à l'utilisateur** de saisir les **4 valeurs** du tableau B.
    - **Fusionne** les deux tableaux dans un **troisième tableau C**, de manière à ce que C contienne tous les éléments de A suivis de ceux de B.
  - Affiche à la fin:
    - le contenu du tableau A,
    - le contenu du tableau B,
    - et le contenu du tableau C après la fusion.

- **Exercice d'application: Solution**

```
import java.util.Scanner;
import java.util.Arrays;
public class FusionTableauxSaisie {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Étape 1 : Déclaration et saisie du tableau A
        int[] A = new int[5];
        System.out.println("--- Saisie du tableau A ---");
        for (int i = 0; i < A.length; i++) {
            System.out.print("Entrez la valeur " + (i + 1) + " : ");
            A[i] = sc.nextInt();
        }
```

```
// Étape 2 : Déclaration et saisie du tableau B
        int[] B = new int[4];
        System.out.println("\n--- Saisie du tableau B ---");
        for (int i = 0; i < B.length; i++) {
            System.out.print("Entrez la valeur " + (i + 1) + " : ");
            B[i] = sc.nextInt();
        }
// Étape 3 : Création du tableau C pour la fusion
        int[] C = new int[A.length + B.length];
        // Copier les éléments du tableau A dans C
        for (int i = 0; i < A.length; i++) {
            C[i] = A[i];}
```

- **Exercice d'application: Solution (suite)**

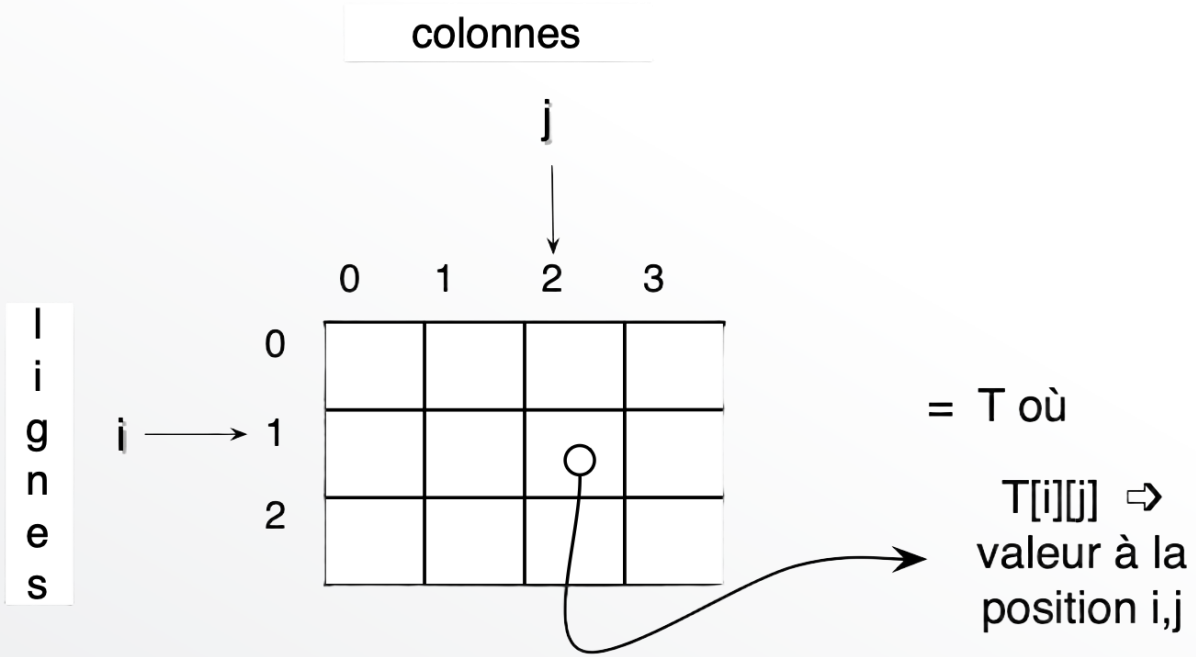
```
// Copier les éléments du tableau B dans C à la suite de A
        for (int i = 0; i < B.length; i++) {
            C[A.length + i] = B[i];
        }
// Étape 4 : Affichage des trois tableaux
        System.out.println("Tableau A : " + Arrays.toString(A));
        System.out.println("Tableau B : " + Arrays.toString(B));
        System.out.println("Tableau C (fusionné) : " + Arrays.toString(C));
        sc.close();
    }
}
```

Partie 4

Matrices

Matrices

- Vu comme une grille composée de lignes et de colonnes,
- Chaque élément est désigné par sa position dans cette grille : (numéro de ligne, numéro de colonne),
- Si T est un tableau à deux dimensions, l'élément à la ligne i et colonne j est donné par T[i][j].



Matrices

- Déclaration : (Type [][] nomMatrice;).

Exemple: int notes[][];

int[][]notes; // forme recommandée en Java

- Création du tableau

notes = new int[3][4]; // crée une matrice de 3 lignes et 4 colonnes

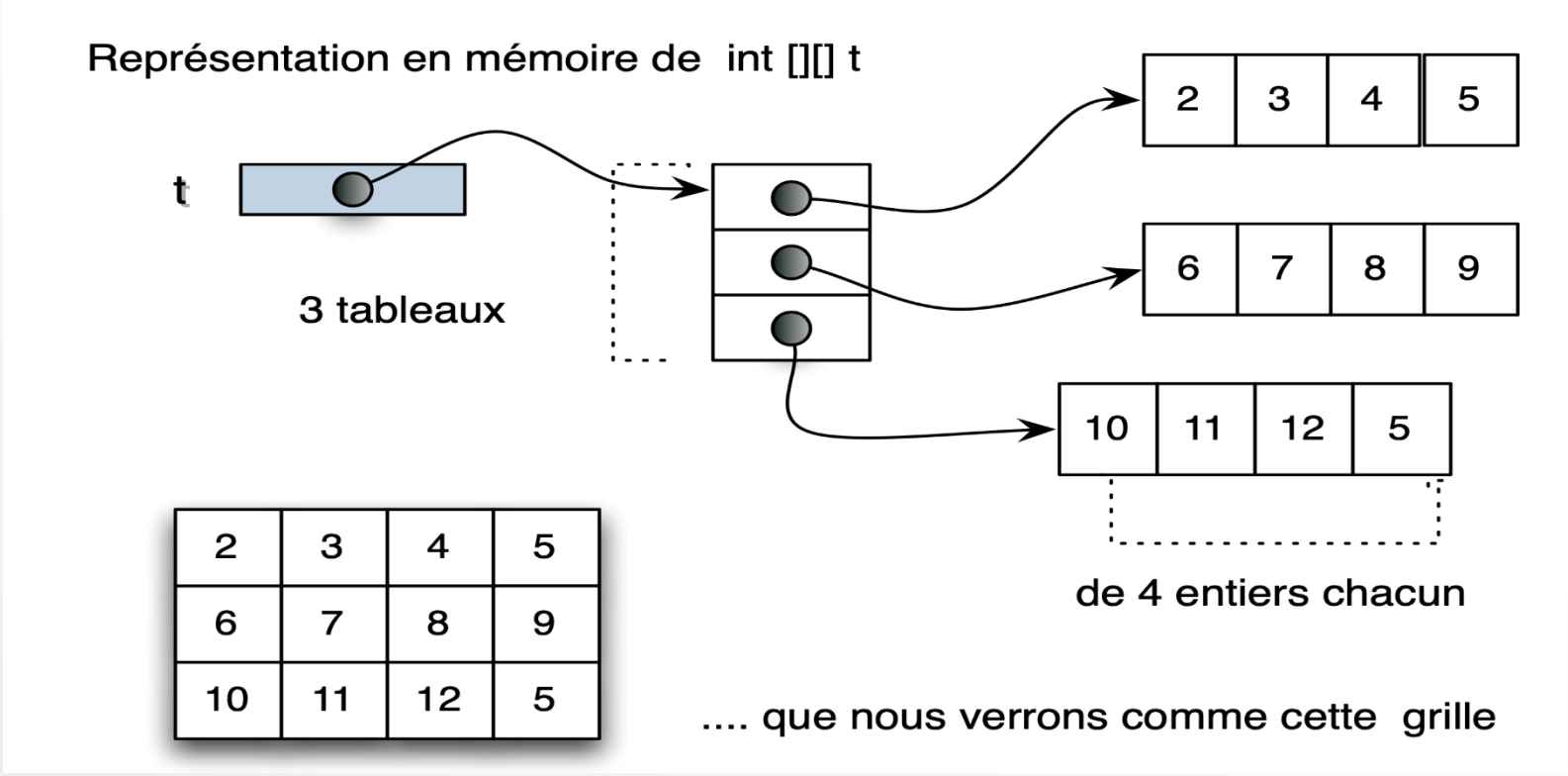
- Déclaration + création en une seule ligne

int[][] notes = new int[3][4];

ou bien, avec initialisation directe: int[][] notes = {{12, 15, 18, 10},{14, 16, 11, 13},{9, 17, 15, 14}};

Représentation de matrices en mémoire

- Une matrice est en réalité un tableau de tableaux.
- Exemple : int[][] t = new int[3][4] est formé de 3 tableaux de int, où chacun de ces 3 tableaux a 4 composantes de type int.



Les Matrices: *Opérations de Base*

- **Parcourir une matrice**
  - Pour parcourir une matrice, on utilise généralement deux boucles for imbriquées.
  - Le nombre de lignes est donné par: **matrice.length**
  - Le nombre de colonnes est donné par: **matrice[i].length**

```
Méthode 1:
public class Demo {
public static void main( String [] args ) {
// Déclaration et initialisation de la matrice
      int[][] M = {{10, 20, 30},{40, 50, 60}};
 // Parcours de la matrice
      for (int i = 0; i < M.length; i++) {
            for (int j = 0; j < M[i].length; j++) {
                        System.out.println(M[i][j]);
         } }}}
```

Les matrices: *Opérations de Base*

- **Parcours avec un for each**
  - Une **matrice est un tableau de tableaux**.
  - La boucle for-each permet de parcourir:
    - chaque **ligne** de la matrice,
    - puis chaque **élément** de la ligne.

```
Méthode 2:
public class Demo {
public static void main( String [] args ) {
// Déclaration et initialisation de la matrice
        int[][] M = {{10, 20, 30},{40, 50, 60}};
        // Parcours de la matrice avec for-each
        for (int[] ligne : M) {          // parcours des lignes
            for (int valeur : ligne) {   // parcours des colonnes
                System.out.println(valeur);
            }}}}
```

Les matrices: *Opérations de Base*

- **Accéder à un élément**

Exemple:

```
int[][] notes = {{12, 15, 18, 10},{14, 16, 11, 13}};

System.out.println(notes[0][0]);   // Résultat : 12

System.out.println(notes[0][2]);   // Résultat : 18
```

- **Modifie un élément**

```
notes[0][1] = 20;   // Remplace 15 par 20
notes[1][3] = 11;   // Remplace 13 par 11
```

Partie 5

Classe String

String: *Définition*

Une chaîne de caractères en Java est un objet de la classe String qui représente une suite de caractères. Elle est **immutable**, c'est-à-dire que son contenu ne peut pas être modifié après sa création.

Exemple:

```
String s = "Test";

s.toUpperCase();

System.out.println(s);
```

```
String s = "Test";

s =s.toUpperCase();

System.out.println(s);
```

String: *Manipulation*

1. *Déclaration:* ***String indentifiant***
2. *Création d'une chaine:* ***String s = "Test"; ou bien String s = new String("Test");***
3. *Longueur d'une chaîne:*

```
String s = "Java";
int longueur = s.length();
System.out.println(longueur); //4
```

4. *Concaténation avec l'opérateur* +

```
String prenom = "Test";
String message = "Bonjour " + prenom;
System.out.println(message);// Bonjour Test
```

String: *Manipulation*

5. Concaténation avec la méthode concat()

```
String s1 = "Bonjour ";
String s2 = "Test";
String resultat = s1.concat(s2);
System.out.println(resultat); //Bonjour Test
```

6. Vérification si une sous-chaîne existe.

```
String s = "Java Programming";

boolean test = s.contains("Java");

System.out.println(test);// true
```

String: *Manipulation*

7. Recherche un caractère ou une sous-chaîne (indice)

```
String s = "Java Programming";
int pos = s.indexOf('a');
System.out.println(pos);//1
```

```
String s = "Java Programming";
int pos = s.indexOf("Pro");
System.out.println(pos);//5
```

8. Vérification si la chaîne n'est pas nulle et ce n'est pas vide.

```
if (s != null && !s.isEmpty()) {
System.out.println("Chaîne valide");}
```

```
String s = "   ";
System.out.println(s.isEmpty());        // false
System.out.println(s.trim().isEmpty());   // true
```

String: *Manipulation*

9. Comparaison des chaînes de caractères en Java: == (Référence)

```
String a = "Java";
String b = "Java";
System.out.println(a == b);   // true
```

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);   // false
```

10. Comparaison des chaînes de caractères en Java: equals(Contenu)

```
String a = "Java";
String b = "JAVA";
System.out.println(a.equals(b));   // false
System.out.println(a.equalsIgnoreCase(b));  // true
```